

Atividade 03

- **Aluno:** Manuel Ferreira Junior
- **Disciplina:** Processamento Digital de Imagens

Questão 01

Para esta questão, considere as imagens:

Pattern1.png

Pattern1_blur.png

Pattern1_blur2.png

Pattern2.png

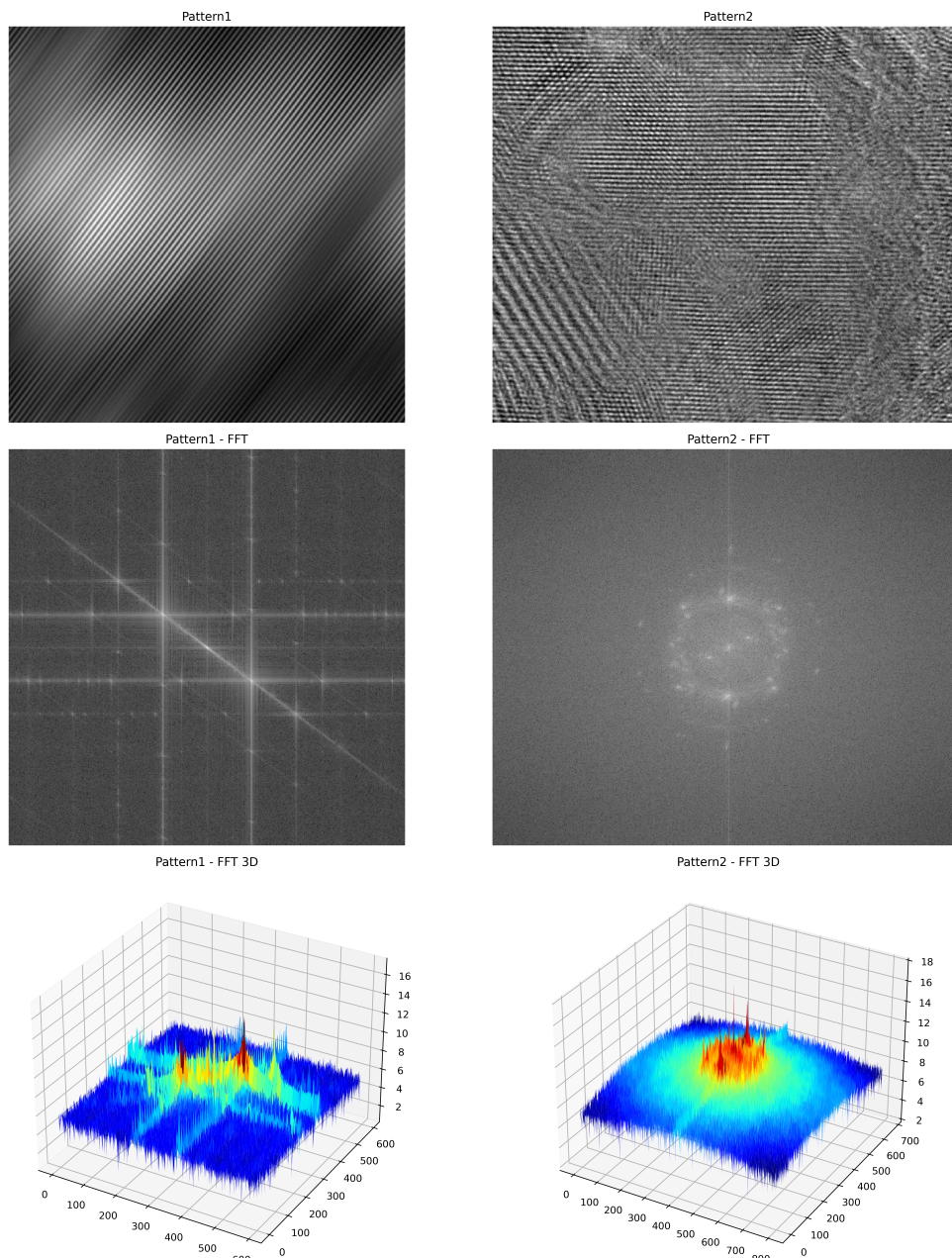
As imagens Pattern1_blur e Pattern1_blur2 são versões embaçadas com diferentes intensidades da imagem Pattern1.

A)

Nas imagens Pattern1 e Pattern2, calcule a magnitude da Transformada de Fourier e disserte sobre esse gráfico em relação às imagens originais, observando o que acontece com as altas e baixas frequências.

R.:

Como pode ser visto na questão, a transformada de Fourier é uma ferramenta essencial para análise da frequência dos contrastes na imagem, podendo ajudar a identificar padrões espaciais distribuídos ao longo da imagem original, a partir de análises de altas e baixas frequências. Nesse caso, para as imagens *Pattern1* e *Pattern2*, é possível observar dois comportamentos bem distintos como visto abaixo



A imagem *Pattern1* apresenta um padrão período, sendo ele definido por traçados diagonais bem definidos e contínuos, indicando uma periodicidade na estrutura da imagem, sendo evidenciado ainda mais ao analisar a FFT da imagem, com picos localizados e simétricos. O padrão regular encontrado na transformada de fourier evidencia o comportamento periódico da imagem original, devido a essa presença de picos e simetrias na magnitude da transformada.

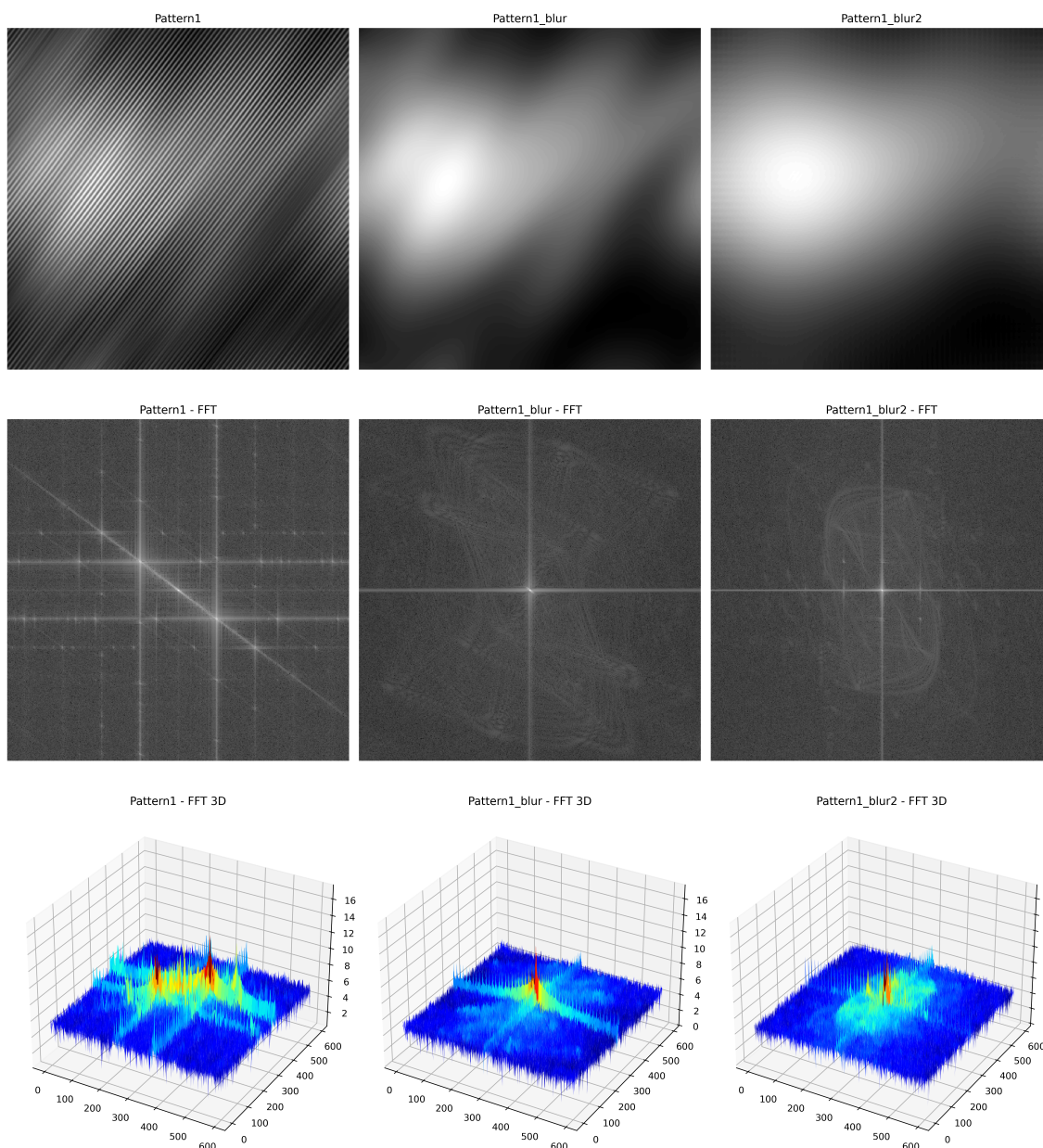
A segunda imagem, *Pattern2*, possui um padrão mais desordenado, comparado a primeira imagem, não tendo um comportamento periódico completo e sim mais complexo. Contudo a imagem ainda que sem periodicidade completa, existe partes da imagem que apresentam alguma periodicidade mesmo que em pouco espaço. Aparentemente, existem uma interseção entre dois comportamentos, um horizontal e outro diagonal na imagem, isso se reflete na transformada de fourier, tendo uma distribuição mais espalhada, contudo com uma região circular no centro, com alguns picos ao longo do perímetro da circunferência. Uma diferença clara entre as transformadas, que pode ser evidenciada mais claramente quando olhamos o plot 3D das transformadas, é a presença de 2 picos bem evidentes na primeira imagem *Pattern1*, enquanto a *Pattern2* apresenta alguns picos próximos dentro da circunferência central.

Comparando ambas as imagens e suas transformadas, podemos notar que uma imagem com um comportamento periódico claro, ao longo da imagem completa (*Pattern1*) apresenta uma representação na magnitude da

transformada de fourier bem definida, com traços simetricos e com picos bem definidos; Já imagens com padrão tão complexos, como a sobreposição de comportamentos horizontais e diagonais, além da alteração da frequência dos tons, devem apresentar uma transformada com um comportamento mais complexos, como no exemplo, uma circunferencia com raio bem definido, e diversos picos ao longo do centro.

B)

Agora, analise o que acontece com a magnitude da Transformada de Fourier à medida que filtros passa baixa vão sendo aplicados na imagem Pattern1 com intensidades cada vez mais fortes. Isso pode ser visto nas imagens Pattern_blur e Pattern_blur2; não é necessário embaçar mais a imagem original.



R.:

Logo apos a primeira aplicação do filtro passa baixa, a imagem já tem seu ruido periodico retirado, os traços horizontais ao longo da imagem desaparecem e ocorre o efeito de borramento esperado apos aplicação do filtro. Analisando a transformada de fourier, é mais nitido a cruz horizontal e vertical na magnitude bem definida, porém algumas altas frequências aparentam estar espalhadas ainda ao longo da matriz de transformada. Um ponto interessante de se ressaltar, é o desaparecimento dos picos de alta frequência anteriores, agora restando

apenas um pico de alta frequência, com alguns outros picos espalhados mas maioria centrado ao longo da cruz. Após aplicação do segundo filtro passa baixa, a cruz permanece ainda com altas frequência, mas ainda sim bem reduzidos comparado a primeira aplicação do filtro, e o comportamento das altas frequências espalhadas agora se encontram mais próxima do centro, ou seja, mais perto do pico de alta frequência da imagem.

Questão 03

Nas imagens coloridas da questão, implemente um algoritmo automático que diminua a quantidade de cores das imagens, agrupando tons semelhantes (por exemplo, uma região de tons avermelhados deve ser tornar uma região com apenas um tom de vermelho). Não deve ser usado dithering. Considere que a imagem final pode ter, aproximadamente, de metade a um terço da quantidade de cores das imagens originais. Cada imagem tem:

- **araras.png**: 112.233 cores;
- **F1.png**: 85.837 cores;
- **green-water.png**: 33.801 cores;
- **surf.png**: 47.229 cores.

O mesmo algoritmo deve ser usado nas 4 imagens. Observações:

1. Se precisar, pode usar conversões entre modelos de cores já implementadas em Python ou qualquer outra linguagem.
2. Não pode usar K-means ou qualquer outra técnica de agrupamento.
3. Só pode usar técnicas vistas na disciplina.
4. Todo o processo deve ser automático sem participação do usuário.

R.:

O algoritmo irá se basear em agrupar cores próximas, considerando faixas de bins definidas entre 0 e 255 para cada canal de cor. Se o valor daquele pixel pertencer a esse intervalo, ele será atribuído o ponto médio do intervalo.

O algoritmo irá realizar uma busca exaustiva entre um número x de bins a um número y de bins, com salto definido como "p". O intuito dessa busca exaustiva é tentar achar o número de bins ideal para que seja satisfeita a condição de que o número total de cores da imagem esteja entre $1/3$ da imagem original a 50% da imagem original.

Vamos definir por partes, primeiro a lógica da implementação, e em seguida a implementação utilizando o python.

[Algoritmo] Aproximação por bins

1. Definir condições de busca:

- **Quantidade de bins:** para cada canal, será feita uma quebra em bins (ex.: 0 a 63, 64 a 127, 128 a 191 e 192 a 255, total de 5 bins). Nesse passo, será definido uma lista de possíveis bins para busca, por exemplo, se selecionar 50 a 256 com salto de 10, teremos o primeiro bin que irá quebrar o intervalo de 0 a 255 em 50 partes, o segundo bin irá quebrar o intervalo de 0 a 255 em 60 partes, o terceiro bin irá quebrar o intervalo de 0 a 255 em 70 partes, e assim por diante; na parametrização da

solução vamos considerar o início igual a 50, e o final da procura em 150 bins, considerando um salto de 10.

- **Parâmetros usados:**

Abaixo temos descrito quais parametros de range de busca foram utilizados.

- `inicio (a)`: 50
- `final (b)`: 150
- `salto (step)`: 10

2. Definição de critério de parada:

- **Critério de parada:** para cada iteração de bins do passo **(1)**, será realizado uma checagem de parada, aonde a condição será definida por:

$$\begin{cases} \text{Parar busca e seleção do } i\text{-ésimo bin } (B = b^{(i)}), & \text{se } \frac{1}{3} \cdot T_o \leq T_r^{(i)} \leq 0.5 \cdot T_o \\ \text{Próxima iteração,} & \text{se } T_r^{(i)} < \frac{1}{3} \cdot T_o \text{ ou } T_r^{(i)} > 0.5 \cdot T_o \end{cases}$$

Sendo:

$$\begin{cases} T_r^{(i)} : & \text{Total de cores da imagem reduzida para o } i - \text{ésimo bin} \\ T_o : & \text{Total de cores da imagem original} \end{cases}$$

Além da checagem para convergência acima para cada iteração, em um caso de não satisfazer a condição definida para nenhum bin, será atribuído aquele que minimize o número de cores, ou seja:

$$B = \min_{a \leq i \leq b, step} \{b^{(i)}\}$$

Sendo:

$$\begin{cases} B : & \text{Bin escolhido ao longo das iterações} \\ b^{(i)} : & i\text{-ésimo bin a ser usado} \\ a : & \text{Número inicial de bins para procura} \\ b : & \text{Número final de bins para procura} \\ step : & \text{Salto a ser dado a cada iteração de um bin para o próximo} \end{cases}$$

3. Processo de calculo para redução:

Obs.: O processo a ser descrito será aplicado para cada canal existente.

Definição: Para cada iteração, o i -ésimo bin, será gerado faixas baseado na média entre `bin[1:]` e `bin[(-1)]`, ou seja:

$$b^{(i)} = [x_0, x_1, \dots, x_n]$$

aonde o novo valor a ser atribuído para cada pixel será definida pelas entradas do vetor abaixo:

$$V = \frac{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}}{2} = \frac{\begin{bmatrix} x_1 + x_0 \\ x_2 + x_1 \\ \vdots \\ x_n + x_{n-1} \end{bmatrix}}{2} = \begin{bmatrix} \frac{x_1 + x_0}{2} \\ \frac{x_2 + x_1}{2} \\ \vdots \\ \frac{x_n + x_{n-1}}{2} \end{bmatrix}$$

Logo, para cada valor novo por pixel para cada canal, teremos a atribuição abaixo:

$$\begin{cases} \frac{x_1 + x_0}{2}, & \text{se } x_0 \leq pixel_{00} \leq x_1 \\ \frac{x_2 + x_1}{2}, & \text{se } x_1 \leq pixel_{00} \leq x_2 \\ \vdots \\ \frac{x_n + x_{n-1}}{2}, & \text{se } x_{n-1} \leq pixel_{00} \leq x_n \end{cases}$$

Isso será feito para cada canal da imagem (Img), pixel a pixel, ou seja, aplicado para cada canal na sua respectiva matriz abaixo:

$$Img^{(c)} = \begin{bmatrix} pixel_{00} & pixel_{01} & \cdots & pixel_{0m} \\ pixel_{10} & pixel_{11} & \cdots & pixel_{1m} \\ \vdots & \vdots & \ddots & \vdots \\ pixel_{n0} & pixel_{n1} & \cdots & pixel_{nm} \end{bmatrix}_c$$

com $c = \{\text{red, green, blue}\}$.

4. Imagem reduzida:

Apartir dos passos anteriores, é definido uma nova imagem, a qual o total de cores deve estar definido no intervalo de 1 / 3 da original a 50% da original.

[Implementação] Aproximação por bins

1. Leitura das imagens:

```
# Caminho para a imagem
images_path = {
    "araras": "/aatv03_lista-final/Q3/araras.png",
    "F1": "/aatv03_lista-final/Q3/F1.png",
    "green-water": "/aatv03_lista-final/Q3/green-water.png",
    "surf": "/aatv03_lista-final/Q3/surf.png",
}

imgs = {}
for key, path in images_path.items():
    img = cv2.imread(path)
    imgs[key] = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

2. Funções:

```

def count_cores(
    image: np.ndarray
) -> int:
    """
    Conta o número de cores únicas em uma imagem RGB.

    Args:
        image (np.ndarray): Imagem RGB.

    Returns:
        int: Número de cores únicas.
    """
    # Converte a imagem para um array 2D
    pixels = image.reshape(-1, 3)
    # Conta as cores únicas
    unique_colors = np.unique(pixels, axis=0)
    return unique_colors.shape[0]

def reduce_colors(
    image: np.ndarray,
    bins: int = 4
) -> np.ndarray:
    """
    Reduz a quantidade de cores de uma imagem RGB, agrupando tons semelhantes.

    Args:
        image (np.ndarray): Imagem RGB.
        bins (int): Número de intervalos para quantização em cada canal (R, G, B).

    Returns:
        np.ndarray: Imagem com cores reduzidas.
    """
    # Calcula os limites dos intervalos
    bin_edges = np.linspace(
        0,
        256,
        bins + 1,
        dtype=np.int32
    )
    bin_centers = (
        (bin_edges[:(-1)] + bin_edges[(1):]) / 2
    ).astype(int)

    # Mapeia cada pixel para o centro do intervalo correspondente
    quantized_image = np.zeros_like(image)
    for i in range(3): # Para cada canal (R, G, B)
        channel = image[:, :, i]
        indices = np.digitize(channel, bin_edges) - 1
        quantized_image[:, :, i] = bin_centers[indices]

    return quantized_image, bin_centers, bin_edges

def adjust_bins(
    image: np.ndarray,

```

```

        list_range: list=[50, 150],
        target_range: tuple = (1/3, 0.5)
    ) -> int:
    """
    Ajusta o número de bins para garantir que a quantidade de cores reduzidas
    esteja dentro do intervalo desejado.

    Args:
        image (np.ndarray): Imagem RGB.
        list_range (list): Intervalo de bins a serem testados (mínimo, máximo).
        target_range (tuple):
            Intervalo desejado (mínimo, máximo) como fração da quantidade de
            cores originais.

    Returns:
        int: Número de bins ajustado.
    """
    original_colors = count_cores(image)
    min_colors = int(original_colors * target_range[0]) # min % do numero de
cores
    max_colors = int(original_colors * target_range[1]) # max % do numero de
cores

    # Testar diferentes números de bins até atingir o intervalo desejado
    for bins in range(*list_range):
        reduced_image, _, _ = reduce_colors(image, bins)
        reduced_colors = count_cores(reduced_image)
        if min_colors <= reduced_colors <= max_colors:
            return bins

    # Caso não encontre, retorna o máximo de bins permitido
    return np.min(list_range)

```

3. Redução de imagem:

```

# Aplicar a redução de cores nas imagens
reduced_imgs = {}
bins_per_image = {}
for key, img in imgs.items():
    # Ajustar dinamicamente os bins para cada imagem
    bin = adjust_bins(
        img,
        list_range=[50, 150, 10], # parametro de procura de bins
        target_range=(1 / 3, 0.5) # parametro de stop
    )#(1/3, 0.5))
    bins_per_image[key] = bin
    reduced_imgs[key] = reduce_colors(img, bins=bin)

```

4. Salvando imagem de resultado:

```

for key, img in reduced_imgs.items():
    img = img[0]

```

```
img_bgr = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
cv2.imwrite(str(path_assets / f"atv03_q03-reduced-{key}.png"), img_bgr)
```

5. Visualização do resultado:

```
# Exibir as imagens originais e reduzidas
fig, ax = plt.subplots(len(reduced_imgs), 2, figsize=(16, 10))

for idx, (key, img) in enumerate(reduced_imgs.items()):
    count_original = count_cores(imgs[key])
    count_reduced = count_cores(img[0])
    count_perc = 100 * (count_reduced / count_original)

    # Imagem original
    ax[idx, 0].imshow(imgs[key])
    ax[idx, 0].set_title(f"{key} - Original ({count_original} cores)")
    ax[idx, 0].axis("off")

    # Imagem reduzida
    ax[idx, 1].imshow(img[0])
    ax[idx, 1].set_title(f"{key} - Reduzida ({count_reduced} cores |
{count_perc:.2f}%)")
    ax[idx, 1].axis("off")

fig.tight_layout()
fig.savefig(path_assets / "atv03_q03-reduced_colors.png", dpi=400)
plt.show()
```

6. Conclusão

A solução proposta é robusta e flexível, permite ao usuário a seleção de um percentual de cores ideal para que a imagem reduzida possua a partir da imagem original. Além disso, a redução foi significativa, e ainda sim as imagens permanecem com seus contrastes, mantendo a qualidade original da imagem.

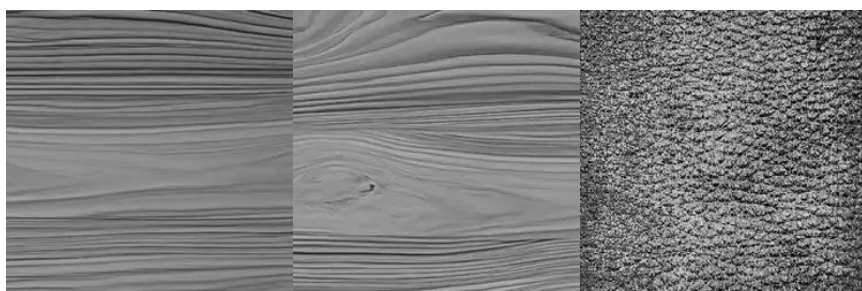
Questão 04

Considere para esta questão as imagens:

Textura1.png

Textura2.png

Textura3.png



Textura 1

Textura 2

Textura 3

As imagens Textura1 e Textura2 são de um mesmo material (suponha Classe 1), tendo sido tiradas de partes diferentes desse material. Já a imagem Textura 3 é de um material diferente (suponha Classe 2). Implemente um algoritmo, automático e com apenas uso de técnicas vistas na disciplina, que consiga classificar essas imagens entre as Classes 1 e 2. Não pode usar técnicas de classificação de aprendizagem de máquina. Manipulações matemáticas ou estatísticas usuais podem ser usadas, sendo devidamente justificadas.

R.:

Solução 1) Coeficiente de variação

Vamos analisar, em conjunto, a variabilidade em torno da média da transformada de fourier, utilizando do **Coeficiente de Variação**, cuja formula pode ser expressa abaixo

$$CV_{\%} = \frac{\sigma}{\mu} \cdot 100$$

aonde

$$\begin{cases} \sigma : \text{Desvio padrão da magnitude da transformada de fourier} \\ \mu : \text{Média da magnitude da transformada de fourier} \end{cases}$$

Para tanto, considerando que o problema é binário, vamos fixar a imagem 1 (*Textura1*) como sendo uma referencia para a classe 1, e um threshold de permissão de 5% de variabilidade sobre essa imagem de referência, para que aquela textura pertença a classe 1. Vamos por partes, abaixo calculamos o desvio padrão para cada Textura.

```
stats_values = {}
for key, value in imgs.items():
    stats_values[key] = {
        "media": float(value["img_fft"].mean()),
        "desvio": float(value["img_fft"].std())
    }
    stats_values[key]["cv"] = 100 * (stats_values[key]["desvio"] /
stats_values[key]["media"])

data_stats = pd.DataFrame(stats_values).round(2)
data_stats
```

	Textura1	Textura2	Textura3
media	6.14	6.60	8.79
desvio	1.19	1.17	0.81
cv	19.43	17.75	9.26

Dado os coeficientes de variação acima, como definido antes, vamos utilizar o coeficiente de variação da imagem *Textura1* como referência, ou seja, 19.43%. Dessa forma, com 5% de permissão de variabilidade sobre o coeficiente de variação, temos o limite superior de 19.43% + 5% = 24.43% e limite inferior de 19.43% - 5% =

14.43%, ou seja, para que a textura seja da classe 1, então o coeficiente de variação da imagem deve estar entre temos definido a regra abaixo:

$$\begin{cases} i \in \text{Classe 1} : \text{ se } 14.43 \leq cv_i \leq 24.43 \\ i \in \text{Classe 2} : \text{ se } cv_i \leq 14.43 \text{ ou } cv_i \geq 24.43 \end{cases}$$

Por fim, temos um método automático para decisão se a textura exibida pertence ao mesmo padrão da 1 e da 2, ou seja, a classe 1.

Este método considera a variabilidade entre os valores de frequências na matriz de magnitudes de fourier, considerando o percentual de variabilidade em torno da média. Dessa forma, o algoritmo leva em consideração a média dos pixels e o desvio padrão da imagem.

Solução 02) Mean Absolute Percentage Error - MAPE

É possível também, utilizando a *Textura1* como padrão para comparação sobre a classe 1, também podemos aplicar calculos matemáticos para comparar uma nova textura com a imagem utilizada. Dessa forma, vamos utilizar a métrica conhecida como *Mean Absolute Percentage Error* (MAPE), para compararmos novas texturas com a escolhida para representar a classe 1. Essa métrica tem como intuito avaliar o quão proximo uma estimativa esta proxima ao valor real; nesse contexto, vamos considerar como uma proximidade, ou seja, quão proximo a 0, mais proximo as frequências de magnitude da transformada de fourier da imagem em questão estão da imagem a ser comparada, apresentando características proximas.

A formula para o MAPE, adaptada para a situação matricial, pode ser expressa abaixo:

$$MAPE_{\%} = 100 \cdot \sum_{j=1}^m \sum_{i=1}^n \frac{|y_{ij} - \hat{y}_{ij}|}{|y_{ij}|}$$

sendo:

$\begin{cases} y_{ij} : i\text{-ésima linha e } j\text{-ésima coluna da matriz de transformada de fourier para a matriz de referência} \\ \hat{y}_{ij} : i\text{-ésima linha e } j\text{-ésima coluna da matriz de transformada de fourier para a matriz de comparação} \end{cases}$

Será necessário considerar também um threshold para garantir um comportamento parecido entre ambas frequências na matriz de magnitude de fourier. Vamos considerar um threshold de 25% para que seja considerado uma imagem similar a de comparação, ou seja, se $MAPE_{\%} < 25\%$, então a imagem pertence a classe 1; caso contrário, pertence a classe 2.

$$\begin{cases} i \in \text{Classe 1} : \text{ se } MAPE_{\%} \leq 25\% \\ i \in \text{Classe 2} : \text{ se } MAPE_{\%} > 25\% \end{cases}$$

Abaixo, vamos calcular esses valores:

```
def mape(y_true, y_pred):
    """Calcula o erro percentual médio absoluto (MAPE)"""
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    return np.mean(np.abs((y_true - y_pred) / y_true)) * 100

img_refer = "Textura1"
data_mapes = {}
for key in ["Textura2", "Textura3"]:
    data_mapes[key] = {
        "mape": mape(
```

```

        imgs[img_refer]["img_fft"].flatten(),
        imgs[key]["img_fft"].flatten()
    )
}
data_mapes = pd.DataFrame(data_mapes).round(2)
data_mapes

```

	Textura2	Textura3
mape	14.65	47.97

Acima, podemos ver que para a *Textura2* o valor de MAPE foi inferior ao threshold escolhido, então podemos afirmar que essa textura pertence a classe 1 também, já a *Textura3*, o valor é superior, logo pela regra sera considerado da classe 2.

Com todas as regras definidas, temos bem claro um algoritmo automático para decisão a qual classe a textura pertence.

Solução 03) Estatística de Komolgorov-Smirnov

Nesta solução, podemos utilizar a estatística calculada para o teste de Komolgorov-Smirnov, aonde o intuito é calcular a distância entre duas distribuições e mensurar quão proximas elas são. A formula é definida abaixo:

$$D = \max_n |F_n(x) - F(x)|$$

Sendo "D" então como a máxima distância entre as distribuições acumuladas empirica amostrada ($F_n(x)$) e a distribuição acumulada empirica a ser comparada ($F(x)$). No contexto atual, vamos considerar como $F_n(x)$ sendo a distribuição acumulada empirica magnitude obtida apartir da transformada de fourier para as imagens *Textura2* e *Textura3* a compararem com a distribuica acumulada empirica da magnitude da transformada de fourier para a imagem *Textura1* ($F(x)$). Para usar essa métrica, também é necessário selecionar um threshold, e dessa forma vamos fixar um valor de até 0.5 para ser considerada da classe 1, caso contrário, será da classe 2, logo:

$$\begin{cases} i \in \text{Classe 1} : \text{ se } D \leq 0.5 \\ i \in \text{Classe 2} : \text{ se } D > 0.5 \end{cases}$$

Abaixo podemos visualizar o proximo, aonde na primeira imagem é feito a comparação da distribuição entre os histogramas das imagens, evidenciando claramente um distribuição afastada para a imagem *Textura3* longe das outras duas. Olhando para a segunda imagem, é possivel ver as distribuições de $F_n(x)$ para todas as texturas, mostrando a diferença de distribuições entre as classes.

```

def ecdf(data):
    """Calcula a distribuição acumulada empírica (ECDF)."""
    sorted_data = np.sort(data)
    cumulative = np.arange(1, len(sorted_data) + 1) / len(sorted_data)
    return sorted_data, cumulative

img_refer = "Textura1"
ecdfs = {}
distances = {}
# Plots
fig, ax = plt.subplots(1, 2, figsize=(20, 8))
for key, img in imgs.items():

```

```

dist = img["img_fft"].flatten()
x, y = ecdf(dist)
ecdfs[key] = {"x": x, "y": y}
# Plotar a Distribuição de Densidade de Kernel (KDE)
sns.kdeplot(dist, label=f"{key} - KDE", fill=True, alpha=0.3, ax=ax[0])

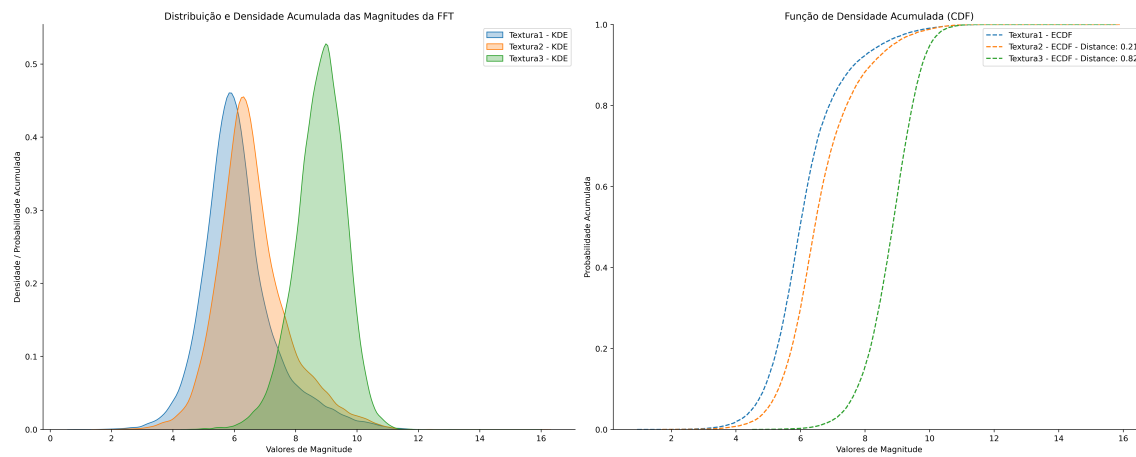
# Calculo da estatística KS
if key != img_refer:
    distances[key] = {
        "ks": float(ks_2samp(imgs[img_refer]["img_fft"].flatten(),
dist).statistic)
    }

# Plotar a Função de Densidade Acumulada (CDF)
if key != img_refer:
    ax[1].plot(
        x,
        y,
        label=f"{key} - ECDF - Distance: {:.2f}".format(key, distances[key]
['ks']),
        linestyle="--"
    )
else:
    ax[1].plot(
        x,
        y,
        label=f"{key} - ECDF",
        linestyle="--"
    )

# Configurar o gráfico
ax[0].set_title("Distribuição e Densidade Acumulada das Magnitudes da FFT")
ax[0].set_xlabel("Valores de Magnitude")
ax[0].set_ylabel("Densidade / Probabilidade Acumulada")
ax[1].set_title("Função de Densidade Acumulada (CDF)")
ax[1].set_xlabel("Valores de Magnitude")
ax[1].set_ylabel("Probabilidade Acumulada")
ax[1].set_ylim(0, 1)
for _ax in ax.flatten():
    _ax.spines[["top", "right"]].set_visible(False)
    _ax.legend()

fig.tight_layout()
fig.savefig(path_assets / "atv03_q04-02.png", dpi=400)
plt.show()

```



Como visto no gráfico, temos um valor de $D = 0.21$ para a *Textura2* e um valor de $D = 0.82$ para a *Textura3*, então pela regra de decisão definida, temos que a textura 2 pertence a classe 1 e a textura 3 pertence a classe 2.

Dessa forma, temos um meio automático para decisão a qual classe pertence a textura testada.